

Linux, Command Line and Git: Beginner Workshop

Duration: about 2 hours. By the end you will be able to move around a terminal, work with files, and use Git and GitHub to track, share and collaborate on a project.

Shell note. All commands in this handout work in Bash or Zsh.

- Windows: use **Git Bash** (installed with Git). Do not use PowerShell or Command Prompt.
 - macOS: use the built-in **Terminal** (Zsh).
 - Linux: use any terminal.
-

0. Before You Start: Get a Terminal

The first two parts of this workshop are about Linux and the command line. You only need a working terminal.

System	What to open
Linux	Any terminal application
macOS	Terminal (Spotlight, then type "Terminal"). Its shell is Zsh.
Windows	Install Git for Windows from https://git-scm.com (keep the default options), then open Git Bash from the Start menu. Git Bash gives you a Unix-style shell, which is what this handout uses. It also installs Git, which we need later.

Test it:

```
echo "hello"  
pwd
```

If it prints **hello** and a folder path, you are ready.

Also create a free account at <https://github.com> ahead of time and verify your email. You will need it in the Git part.

1. What Is Linux?

- **Kernel:** the core of the operating system. It manages hardware, memory and processes. The Linux kernel was started by Linus Torvalds in 1991.
- **Distribution (distro):** a kernel plus tools, a package manager and a desktop. Examples: Ubuntu, Debian, Fedora, Arch.
- **Where it runs:** most web servers, cloud infrastructure, Android phones, routers, supercomputers and embedded devices.
- **Why you care:** macOS is Unix-based, Windows can run Linux through WSL, and developer tools assume a Unix-style terminal. Skills you learn here transfer almost everywhere.

1.1 Filesystem layout

Linux has a single tree that starts at `/` (the root).

Path	Contents
<code>/</code>	Root of everything
<code>/home/<user></code>	Your personal files (<code>~</code> is shorthand for this)
<code>/etc</code>	System configuration files
<code>/usr</code>	Installed programs and libraries
<code>/bin,</code> <code>/sbin</code>	Essential commands
<code>/var</code>	Variable data such as logs
<code>/tmp</code>	Temporary files
<code>/dev</code>	Devices, represented as files

Linux paths use forward slashes (`/`) and are **case-sensitive**: `Notes.txt` and `notes.txt` are different files.

1.2 Users and permissions

- Every file has an owner, a group, and permissions for owner, group and others.

- `sudo <command>` runs one command as the administrator. Use it only when you understand what the command does.
- Package managers install software from trusted repositories:

```
sudo apt install tree    # Debian/Ubuntu
brew install tree       # macOS (Homebrew)
```

2. The Command Line

A **terminal** is the window. The **shell** (Bash or Zsh) is the program inside it that reads your commands.

Anatomy of a command:

```
command -options arguments
ls -l /home
```

2.1 Where am I? Moving around

```
pwd          # print working directory (where am I?)
ls           # list files
ls -l        # long format: permissions, size, date
ls -a        # include hidden files (names starting with .)
ls -la       # both
cd Documents  # go into a folder
cd ..        # go up one level
cd ~         # go to your home folder
cd -         # go back to the previous folder
cd           # no argument also goes home
```

Special path symbols:

Symbo	Meaning
.	The current folder
..	The parent folder
~	Your home folder

/ The root of the
 filesystem

- **Absolute path:** starts from / (for example `/home/sam/notes`). It works from anywhere.
- **Relative path:** starts from where you are now (for example `notes/todo.txt`).

2.2 Creating, copying, moving and deleting

```
mkdir project          # make a folder
mkdir -p a/b/c         # make nested folders in one go
touch notes.txt        # create an empty file (or update its timestamp)
cp notes.txt backup.txt # copy a file
cp -r project project-copy # copy a folder (recursive)
mv notes.txt docs/     # move a file into a folder
mv notes.txt ideas.txt # rename a file
rm backup.txt          # delete a file
rm -r project-copy     # delete a folder and its contents
```

Warning: `rm` does not use a trash can. Deleted files are gone. Never run `rm -rf` on a path you have not double-checked.

2.3 Viewing and writing file contents

```
cat notes.txt          # print the whole file
less notes.txt         # scroll through a file (q to quit)
head -n 5 notes.txt    # first 5 lines
tail -n 5 notes.txt    # last 5 lines
wc -l notes.txt        # count lines
```

Writing text without an editor:

```
echo "Hello" > hello.txt    # > creates the file or OVERWRITES it
echo "Second line" >> hello.txt # >> appends to the end
```

> replaces the whole file without asking. Use >> when you want to add.

For real editing, use any text editor you like (VS Code, Notepad, `nano`). Terminal editors such as `vim` can trap beginners. If you get stuck in vim, press `Esc`, then type `:q!` and press Enter.

2.4 Searching and combining commands

```

grep "error" log.txt      # lines containing "error"
grep -i "error" log.txt   # ignore upper/lower case
grep -n "error" log.txt   # show line numbers
grep -r "TODO" .          # search all files under the current folder
find . -name "*.txt"      # find files by name

```

Pipes (|) send the output of one command into the next:

```

ls | wc -l                # how many items in this folder?
cat log.txt | grep "error" | wc -l
history | grep git         # find earlier git commands you typed

```

Redirection sends output to a file:

```

ls -l > listing.txt
grep "error" log.txt >> errors.txt

```

2.5 Getting help and working faster

```

ls --help                 # short help for most commands
man ls                    # full manual (q to quit; not available in Git Bash)

```

Shortcut	Effect
Tab	Autocomplete file and command names (press twice for options)
Up / Down arrows	Browse earlier commands
Ctrl + R	Search command history
Ctrl + C	Stop the running command
Ctrl + L or clear	Clear the screen
Ctrl + A / Ctrl + E	Jump to start / end of the line

2.6 Permissions (quick look)

```

ls -l script.sh
# -rwxr-xr-- 1 sam staff 120 Oct 2 10:00 script.sh

```

Read `-rwxr-xr--` as: file type, then **owner** `rwx`, **group** `r-x`, **others** `r--`. `r` is read, `w` is write, `x` is execute.

```
chmod +x script.sh    # make a file executable
./script.sh           # run it
```

2.7 CLI exercise

```
cd ~
mkdir git-journey
cd git-journey
touch 1.txt 2.txt
echo "First file" > 1.txt
echo "Second file" > 2.txt
echo "Hello" > greeting.txt
ls -la
cat 1.txt
grep -r "file" .
```

Keep this folder. It becomes your Git project in the next sections.

3. Git

3.0 Git Setup (once per machine)

Install Git (Windows users already have it from Git for Windows):

System	How
macOS	Run <code>git --version</code> . If Git is missing, macOS offers to install the developer tools. Or: <code>brew install git</code>
Debian / Ubuntu	<code>sudo apt install git</code>
Fedora / RHEL	<code>sudo dnf install git</code>

Arch `sudo pacman -S git`

Verify:

`git --version`

Expected: `git version 2.x.x`. If you see `command not found` (or "not recognized"), Git is not installed or not on your PATH. Close and reopen the terminal, then retry.

Configure your identity:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

Check it:

`git config --global --list`

Use the same email you registered on GitHub (or your GitHub noreply address) so commits are linked to your account.

Authentication for GitHub (needed to push). GitHub does not accept your account password for Git operations. Pick one:

- **Windows:** Git Credential Manager is included with Git for Windows. On your first push a browser window opens. Sign in and approve.
- **macOS / Linux, recommended:** install the GitHub CLI (<https://cli.github.com>), then:

```
gh auth login
gh auth setup-git
```

- **Any system, alternative:** create a Personal Access Token on GitHub (Settings, Developer settings, Personal access tokens) with repository access. When Git asks for a password, paste the token.

Git Concepts

3.1 Why version control?

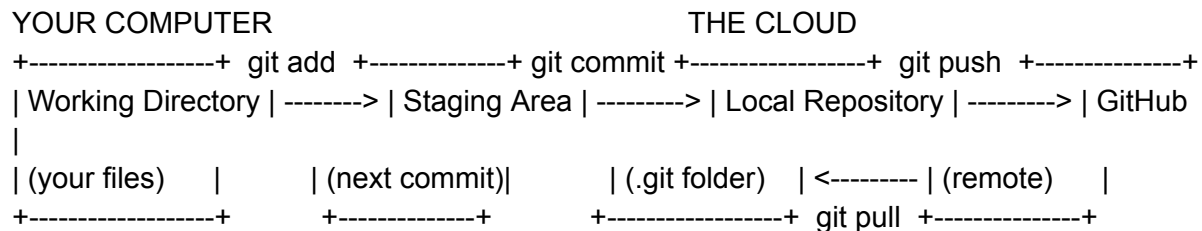
Without it you end up with `report_final_v2_FINAL_really.docx`. Version control records the history of your project so you can see what changed, who changed it and why, and go back if something breaks.

3.2 Git vs GitHub

	Git	GitHub
What it is	Version control tool (open source)	Website for hosting Git repositories
Where it runs	On your computer	On GitHub's servers (owned by Microsoft)
Created by	Linus Torvalds (2005)	GitHub, Inc. (2008)
Main job	Track changes and history	Backup, collaboration, reviews, Pull Requests
Alternatives	None (it is the standard)	GitLab, Bitbucket

Git is the engine. GitHub is one place to store and share what Git tracks.

3.3 The four places your work lives



- **Working directory:** the files you see and edit.
- **Staging area:** the changes you have chosen to include in the next snapshot.
- **Local repository:** the hidden `.git` folder that stores every committed snapshot.
- **Remote repository:** the copy on GitHub.

Git stores **snapshots** of your project at each commit, not just a list of edits. A **commit** is a snapshot with an author, a timestamp, a message and a unique ID (a SHA).

3.4 File states

State	Meaning
-------	---------

Untracked	Git sees the file but has never been told to track it
Modified	A tracked file changed but is not staged
Staged	Marked for inclusion in the next commit
Committed	Safely stored in the local repository

4. The Core Local Loop

4.1 Create the repository

```
cd ~/git-journey
git init
```

Output: `Initialized empty Git repository in .../git-journey/.git/`

Git created a hidden `.git` folder. See it:

```
ls -la
```

Run `git init` once, at the root of your project. Do not run it inside your home folder or inside a folder that is already a repository. If you are unsure, run `git status`.

4.2 Check the state

```
git status
```

You will see `1.txt`, `2.txt` and `greeting.txt` listed as **untracked**. Run `git status` often. It is the most useful Git command.

4.3 Stage changes

Command	What it stages
<code>git add <file></code>	One specific file

<code>git add <file1></code>	Several named files
<code><file2></code>	
<code>git add .</code>	All new, modified and deleted files in the current folder and below
<code>git add -A</code>	All changes in the whole repository
<code>git add *.txt</code>	Files matching the pattern (the shell expands it before Git sees it)

About `git add *`: the shell expands `*` into a list of names. It skips hidden files (names starting with `.`) and does not include files you deleted, so it can miss changes. Prefer `git add ..`

```
git add 1.txt
git status      # 1.txt is now staged
git add .
git status      # everything is staged
```

Unstage a file without losing your edits:

```
git restore --staged 2.txt
```

4.4 Commit

A commit records what is in the staging area. Think of staging as checking yourself in the mirror before leaving the house. You only commit what you have deliberately staged.

```
git commit -m "Add initial text files"
```

Good commit messages are short, in the imperative mood and specific: `Add login form validation`, not `stuff`.

If you run `git commit` without `-m`, Git opens a text editor. If you end up in vim, press `Esc`, type `:q!` and Enter to leave, then retry with `-m`.

4.5 See what changed

```
echo "More text" >> 1.txt
git status      # 1.txt is modified
git diff        # changes not yet staged
git add 1.txt
git diff --staged # changes staged but not committed
git commit -m "Extend first file"
```

4.6 Read the history

```
git log          # full history
git log --oneline # one line per commit
git log --oneline --graph --all # with branch structure
git show <commit-id> # details of one commit
```

Press **q** to leave the log view.

4.7 Ignore files you do not want tracked

Create a file named **.gitignore**:

```
# one pattern per line
*.log
node_modules/
.env
```

Never commit passwords, API keys or **.env** files. If a secret reaches GitHub, treat it as compromised and replace it.

4.8 Remove and rename files

```
git rm old.txt          # delete the file and stage the deletion
git rm --cached notes.txt # stop tracking, keep the file on disk
git mv a.txt b.txt       # rename and stage
```

5. Branches

A **branch** is an independent line of work. The default branch is **main**. Imagine a main kitchen serving customers and a test kitchen where you try new recipes. When the recipe is good, you bring it into the main kitchen.

```
git branch          # list branches (* marks the current one)
git switch -c feature-greeting # create and switch to a new branch
git switch main      # go back to main
git branch -d feature-greeting # delete a merged branch
```

Watch your file explorer while switching: files change to match the branch you are on.

5.1 Merging

```
git switch main
git merge feature-greeting
```

If the branches changed different parts of the project, Git merges automatically. If both changed the same lines, you get a conflict.

5.2 Resolving a merge conflict

Git pauses the merge and writes markers into the file:

```
<<<<<< HEAD
Hello from main
=====
Hello from feature
>>>>>> feature-greeting
```

- Between <<<<<< HEAD and =====: your current branch.
- Between ===== and >>>>>>: the branch you are merging in.

To resolve:

1. Open the file and edit it to the version you want.
2. Delete all three marker lines.
3. Save, then:

```
git add greeting.txt
git commit          # accept the default merge message
```

To abandon a merge and start over: `git merge --abort`.

6. Working with GitHub

6.1 Create a remote repository

1. On github.com, click **New repository**.
2. Name it `git-journey`, set it to **Public**.

3. Leave **Add a README**, **.gitignore** and **license** unchecked.
4. Click **Create repository** and copy the HTTPS URL.

6.2 Connect and push

```
git remote add origin https://github.com/YOUR-USERNAME/git-journey.git
git remote -v          # confirm the connection
git branch -M main     # make sure your branch is named main
git push -u origin main
```

- **origin** is the nickname for the remote URL.
- **-u** links your local **main** to **origin/main**, so later you can just run **git push** and **git pull**.

Refresh the GitHub page. Your files and commits should be there.

6.3 Download a project: clone

```
git clone https://github.com/USER/REPO.git
cd REPO
```

clone downloads the files, the full history and sets up the remote for you.

6.4 Fetch vs pull

Command	What it does
git fetch	Downloads new commits from the remote into origin/main . Does not change your files.
git pull	Runs git fetch and then merges into your current branch. Updates your files.
git fetch	
git log --oneline main..origin/main	# what is new on the remote?
git pull	

6.5 Pull Requests (PRs)

A Pull Request asks the team to review your branch and merge it into **main**.

```
git switch -c readme-update
echo "# git-journey" > README.md
git add README.md
```

```
git commit -m "Add README"
git push -u origin readme-update
```

On GitHub:

1. Click **Compare and pull request** (or go to the **Pull requests** tab, then **New pull request**).
2. Set **base: main** and **compare: readme-update**.
3. Review the diff: green lines are additions, red lines are deletions.
4. Add a title and description, then click **Create pull request**.
5. After review, click **Merge pull request** and confirm.

Update your local copy:

```
git switch main
git pull
git branch -d readme-update
```

7. Undoing Things

Situation	Command	Effect
Discard edits to a file (not committed)	<code>git restore <file></code>	File returns to the last committed version. Uncommitted edits are lost.
Unstage a file	<code>git restore --staged <file></code>	Edits stay in your working directory
Undo the last commit, keep the edits	<code>git reset HEAD~</code>	Commit removed, files unchanged
Undo a commit safely on shared history	<code>git revert <commit></code>	Creates a new commit that cancels the old one
Throw away commits and edits	<code>git reset --hard <commit></code>	Destructive. Uncommitted work is lost permanently

Rules of thumb:

- If the commit has already been pushed and others may have it, use `git revert`, not `git reset`.
 - Work that was committed can usually still be recovered with `git reflog`, which lists where `HEAD` has been. Work that was never committed cannot.
 - Commit often. Committed work is much harder to lose.
-

8. Hands-On Lab

Work through these steps in order. Do not skip the verification lines.

Part A: Local repository

If you completed section 2.7 and section 4, you already have `~/git-journey`. For a clean start use a new folder:

```
cd ~
mkdir git-lab
cd git-lab
git init
echo "Hello" > greeting.txt
echo "Notes" > notes.txt
git status
git add .
git commit -m "Initial commit"
git log --oneline
```

Check: one commit appears in the log.

Part B: Branches and a conflict

```
git switch -c feature
echo "Hello from feature" > greeting.txt
git add greeting.txt
git commit -m "Change greeting on feature"
```

```
git switch main
echo "Hello from main" > greeting.txt
git add greeting.txt
git commit -m "Change greeting on main"
```

```
git merge feature
```

Git reports a conflict. Run `git status`, open `greeting.txt`, and resolve it. For example, replace the contents with a single line:

```
echo "Hello from both" > greeting.txt
git add greeting.txt
git commit -m "Merge feature into main"
git log --oneline --graph --all
```

Check: the graph shows two lines of work joining in one merge commit.

Part C: GitHub

1. Create an empty public repository named `git-lab` on GitHub (no README).
2. Connect and push:

```
git remote add origin https://github.com/YOUR-USERNAME/git-lab.git
git branch -M main
git push -u origin main
```

Check: refresh GitHub and see `greeting.txt` and `notes.txt`.

Part D: Pull Request

```
git switch -c add-readme
echo "# git-lab" > README.md
git add README.md
git commit -m "Add README"
git push -u origin add-readme
```

Open a Pull Request from `add-readme` into `main` on GitHub, then merge it.

```
git switch main
git pull
git log --oneline
```

Check: `README.md` now exists locally and the log includes the README commit and the merge.

Part E: Edit on GitHub, then pull

1. On GitHub, open `README.md`, click the pencil icon, add a line, and commit the change.
2. In your terminal:

```
git pull
cat README.md
```

Check: your local README shows the new line, and `git status` says `nothing to commit, working tree clean`.

9. Command Quick Reference

Terminal

Command	Purpose
<code>pwd</code>	Show current folder
<code>ls -la</code>	List all files with details
<code>cd <path></code>	Change folder
<code>mkdir <name></code>	Create folder
<code>touch <file></code>	Create empty file
<code>cp <src> <dst></code>	Copy
<code>mv <src> <dst></code>	Move or rename
<code>rm <file> / rm -r <dir></code>	Delete (permanent)
<code>cat, less, head, tail</code>	View file contents
<code>grep <text> <file></code>	Search inside files
<code>find . -name "*.txt"</code>	Search for files

<code>\ , >, >></code>	Pipe, overwrite, append
<code>chmod +x <file></code>	Make executable
<code><cmd> --help</code>	Help

Git: setup and basics

Command	Purpose
<code>git config --global user.name "..."</code>	Set your name
<code>git config --global user.email "..."</code>	Set your email
<code>git config --global init.defaultBranch main</code>	New repos start on <code>main</code>
<code>git init</code>	Start a repository
<code>git clone <url></code>	Download a repository
<code>git status</code>	Show file states
<code>git add <file> / git add .</code>	Stage changes
<code>git commit -m "msg"</code>	Save a snapshot
<code>git diff / git diff --staged</code>	See changes
<code>git log --oneline</code>	Compact history
<code>git show <commit></code>	Details of one commit
<code>git rm <file> / git rm --cached <file></code>	Delete / untrack
<code>git mv <old> <new></code>	Rename

Git: branches and remotes

Command	Purpose
---------	---------

<code>git branch</code>	List branches
<code>git switch -c <name></code>	Create and switch
<code>git switch <name></code>	Switch branch
<code>git merge <name></code>	Merge into current branch
<code>git branch -d <name></code>	Delete merged branch
<code>git remote add origin <url></code>	Link a remote
<code>git remote -v</code>	Show remotes
<code>git push -u origin <branch></code>	Upload (first time)
<code>git push / git pull</code>	Upload / download
<code>git fetch</code>	Download without merging

Git: undo

Command	Purpose
<code>git restore <file></code>	Discard uncommitted edits to a file
<code>git restore --staged <file></code>	Unstage
<code>git reset HEAD~</code>	Undo last commit, keep edits
<code>git revert <commit></code>	Undo a commit with a new commit
<code>git reset --hard <commit></code>	Destructive reset
<code>git reflog</code>	Find lost commits

10. Troubleshooting

Problem or message	Cause	Fix
--------------------	-------	-----

<code>git: command not found</code>	Git is not installed or the terminal is stale	Install Git, close and reopen the terminal
<code>Author identity unknown / Please tell me who you are</code>	Name and email are not configured	Run the two <code>git config --global</code> commands from section 3.0
<code>fatal: not a git repository</code>	You are not inside a repository	<code>cd</code> to your project folder, or run <code>git init</code>
<code>nothing to commit, working tree clean</code>	Nothing is modified or staged	Edit a file, then <code>git add</code> it
Committed the wrong thing	Mistake in the last commit	<code>git reset HEAD~</code> if not pushed, otherwise <code>git revert</code>
<code>error: src refspec main does not match any</code>	No commit exists yet, or your branch is not called <code>main</code>	Make a commit, and run <code>git branch -M main</code>
<code>remote origin already exists</code>	You ran <code>git remote add</code> twice	<code>git remote set-url origin <url></code>
<code>Authentication failed / 403</code>	GitHub does not accept account passwords	Use the login steps in section 3.0
<code>rejected ... fetch first or non-fast-forward</code>	The remote has commits you do not have	<code>git pull</code> , resolve any conflict, then <code>git push</code>
<code>Your local changes would be overwritten</code>	Uncommitted edits block a branch switch or pull	Commit them, or run <code>git stash</code> , switch, then <code>git stash pop</code>
Stuck in a text editor after <code>git commit</code>	You omitted <code>-m</code>	In vim: <code>Esc</code> , then <code>:q!</code> . In nano: <code>Ctrl+X</code>
Merge conflict	Same lines changed in two branches	See section 5.2
Typed a secret into a commit	It is now in history	Revoke and replace the secret immediately. Then ask for help cleaning history

11. Where To Go Next

- Practice daily: put your own notes or code in a repository and commit as you work.
- Learn next: `git stash`, `git rebase`, `.gitignore` patterns, SSH keys for GitHub, GitHub Issues and Actions.
- References: <https://git-scm.com/doc> (the free *Pro Git* book), <https://docs.github.com>, and `git help <command>`.